

LEZIONI 18 - 21

CLASS DIAGRAM 6

Associazioni, Aggregazioni, Composizioni e Dipendenze

Ingegneria del Software e Progettazione Web
Università degli Studi di Tor Vergata - Roma

Guglielmo De Angelis
guglielmo.deangelis@iasi.cnr.it

class diagram

- struttura statica del sistema:
 - **nodi** + relazioni

class diagram

- struttura statica del sistema:
 - **nodi** + relazioni
- un nodo modella una **classe** (o una interfaccia) che rappresenta:
 - una entità del dominio
 - elementi che non fanno parte del dominio ma utili nell'ingegnerizzazione del sistema

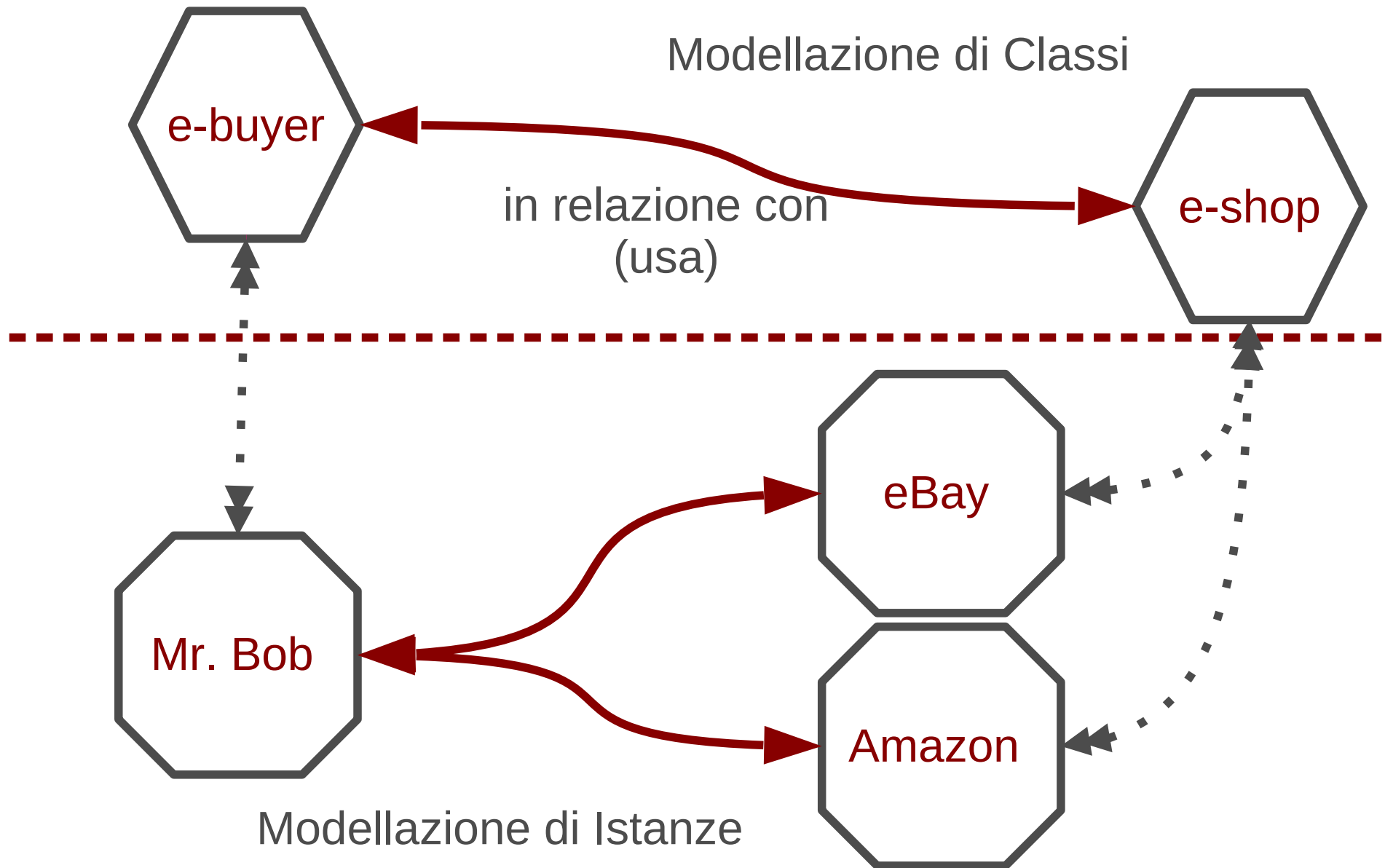
le relazioni in un class diagram

- una relazione rappresenta una “interazione” tra gli elementi modellati
 - una *relazione* tra una classe A ed una classe B significa che A è a conoscenza di B e (in qualche *modo*) può interagirvi
- il tipo di una relazione definisce il “modo di interazione” tra gli elementi che essa coinvolge
- nei class diagram si specificano relazioni tra :
 - classi, oggetti, interfacce, package, etc...

ricordiamo che:

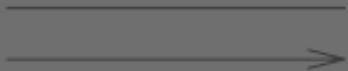
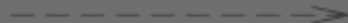
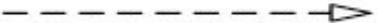
- **operazione**: specifica di un “*segnatura*” (prototipo) che un oggetto mette a disposizione di altri oggetti
- **metodo**: implementazione vera e propria dell'operazione di un oggetto
- **messaggio**: è la richiesta di invocazione a run-time da parte di un oggetto $\mathcal{A}$ su un metodo fornito da un oggetto $\mathcal{B}$

classi **VS.** oggetti



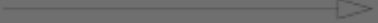
le relazioni in un class diagram

Table 7.2 Graphic paths included in structure diagrams

<i>PATH TYPE</i>	<i>NOTATION</i>	<i>REFERENCE</i>
Aggregation		See "AggregationKind (from Kernel)" on page 38.
Association		See "Association (from Kernel)" on page 39.
Composition		See "AggregationKind (from Kernel)" on page 38.
Dependency		See "Dependency (from Dependencies)" on page 62.
Generalization		See "Generalization (from Kernel, PowerTypes)" on page 71.
InterfaceRealization		See "InterfaceRealization (from Interfaces)" on page 89.

le relazioni in un class diagram

Table 7.3 - Graphic paths included in structure diagrams

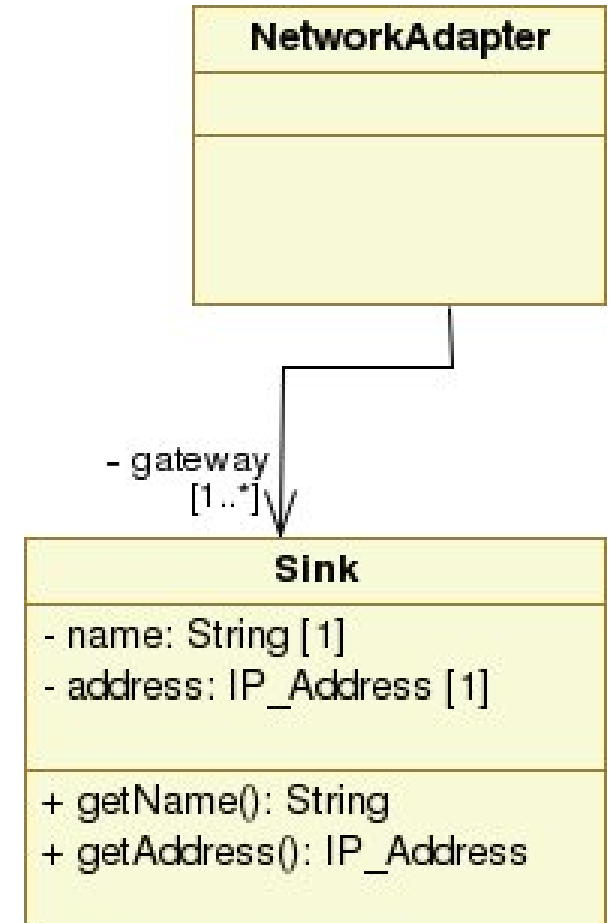
<i>PATH TYPE</i>	<i>NOTATION</i>	<i>REFERENCE</i>
Aggregation		See "AggregationKind (from Kernel)" on page 38.
Association		See "Association (from Kernel)" on page 39.
Composition		See "AggregationKind (from Kernel)" on page 38.
Dependency		See "Dependency (from Dependencies)" on page 62.
Generalization		See "Generalization (from Kernel, PowerTypes)" on page 71.
InterfaceRealization		See "InterfaceRealization (from Interfaces)" on page 89.

associazione

- relazione strutturale tra oggetti di classi differenti
 - può essere simmetrica (navigabile nelle due direzioni)
- una associazione può essere riflessiva, ovvero tra oggetti della stessa classe
- binaria o N-aria (è poco usata)
- rappresentata mediante una linea continua che collega le classi in relazione
- di fatto: indica la possibilità che una classe possa inviare messaggi a quelle associate

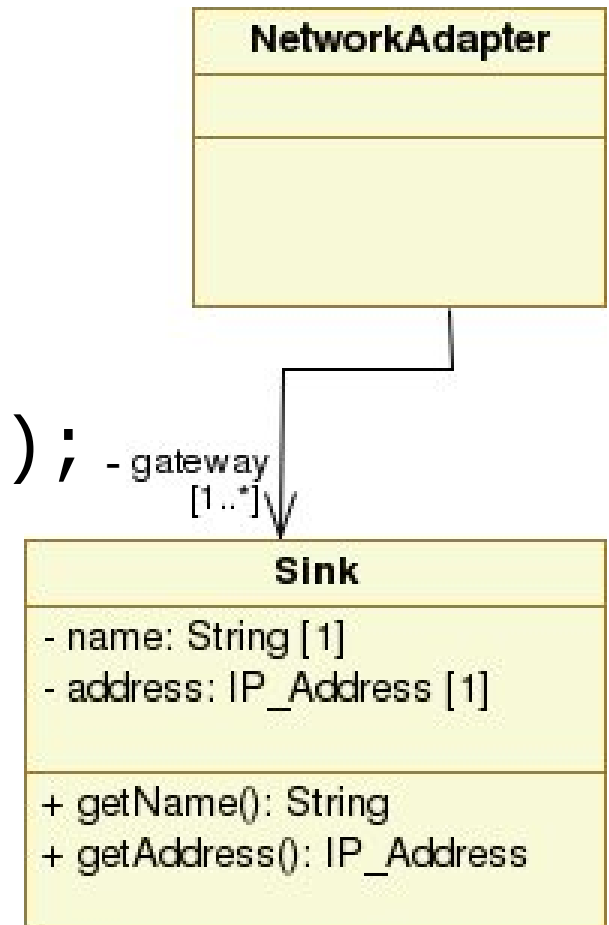
associazione – map in Java

- un associazione può avere:
 - un nome,
 - il ruolo degli operandi,
 - stereotipo
 - cardinalità



associazione – map in Java

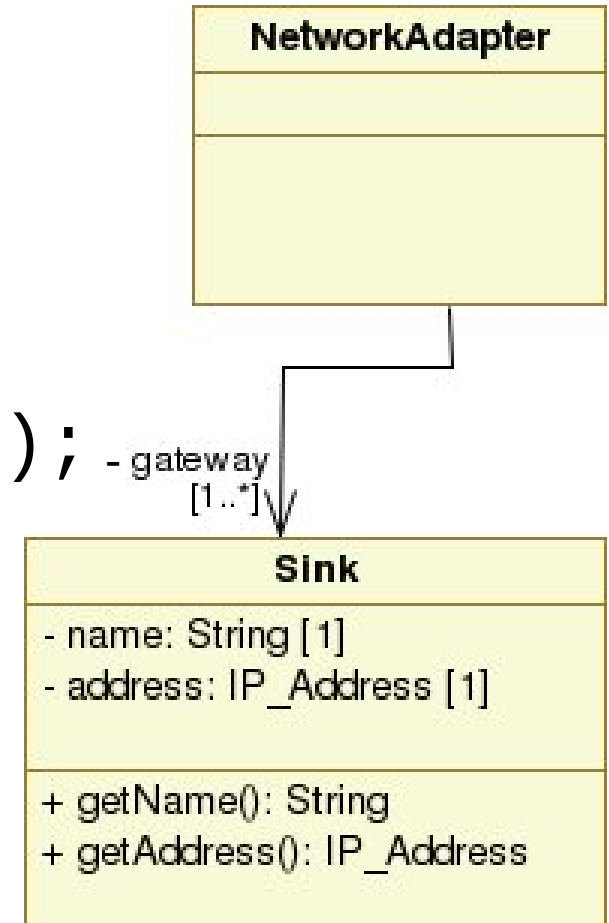
```
public class Sink {  
    private String name;  
    private IP_Address address;  
    public String getName();  
    public IP_Address getAddress();  
}
```



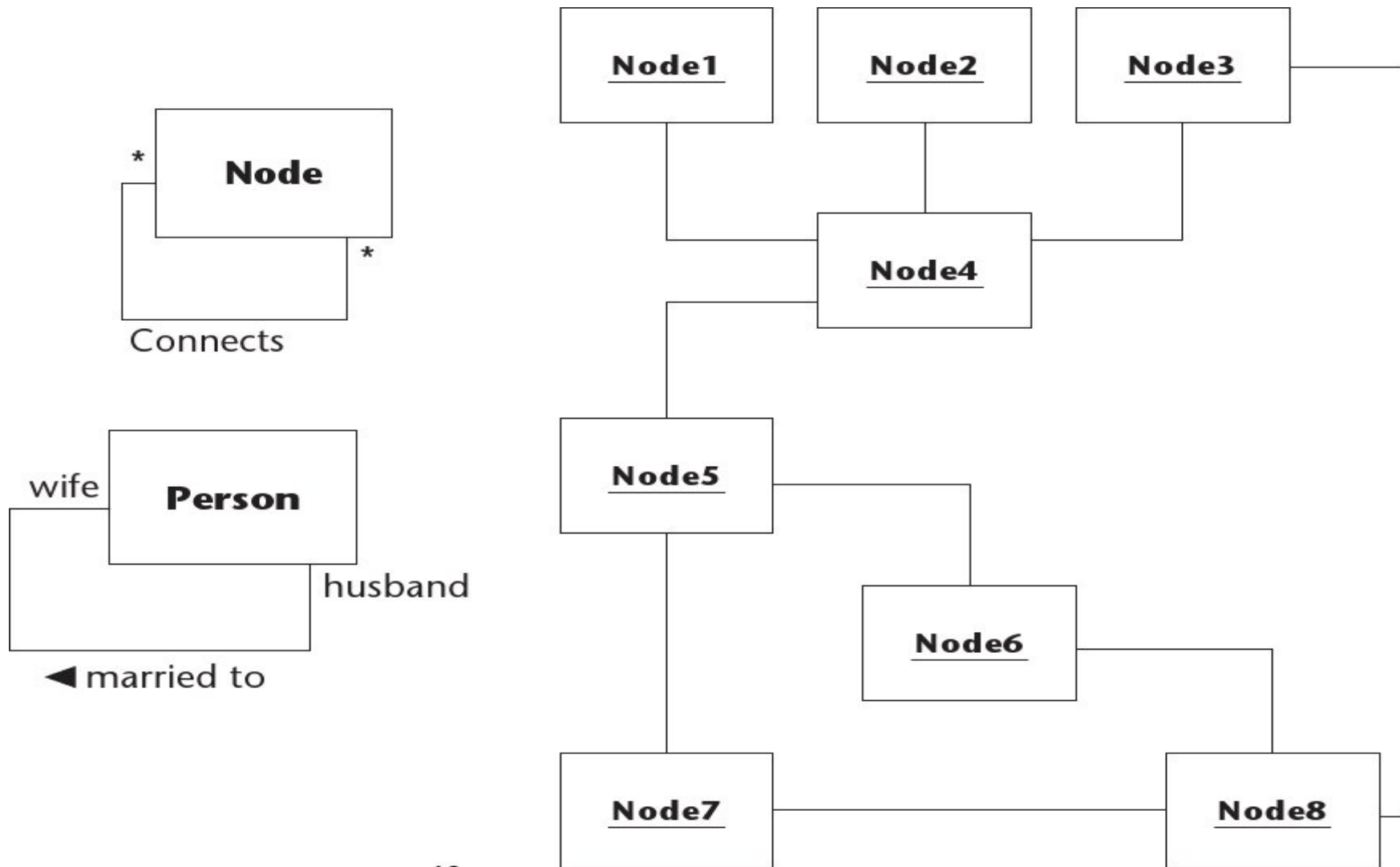
associazione – map in Java

```
public class Sink {  
    private String name;  
    private IP_Address address;  
    public String getName();  
    public IP_Address getAddress();  
}
```

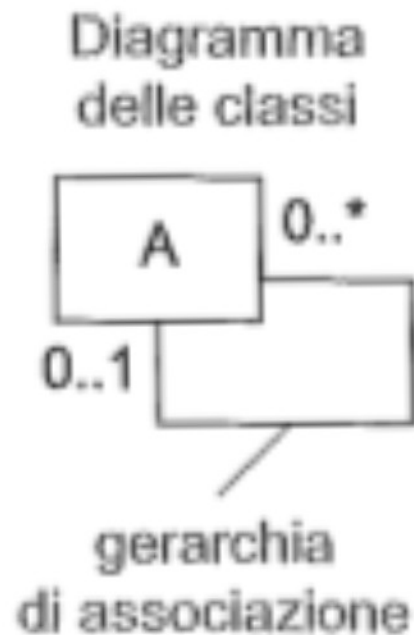
```
public class NetworkAdapter {  
    private Vector<Sink> gateway;  
}
```



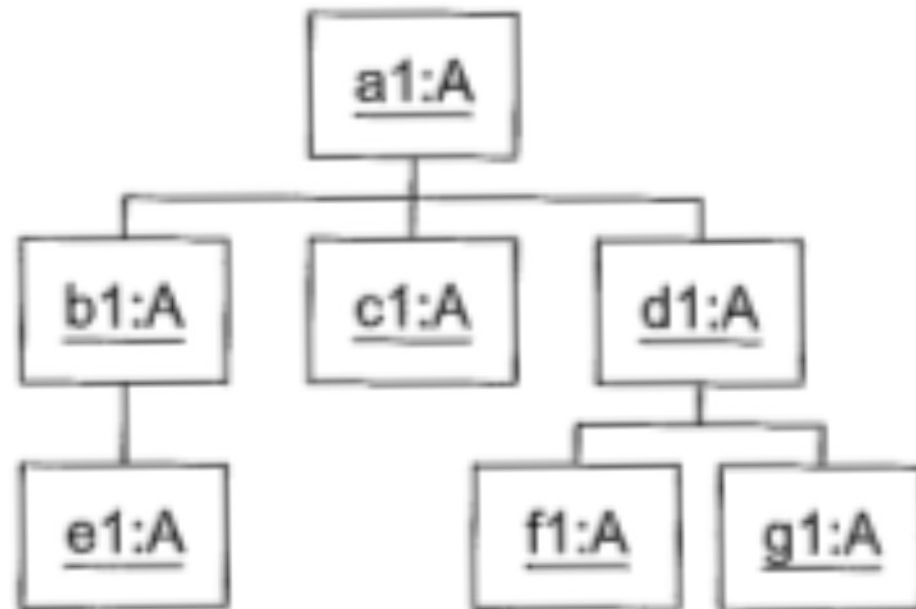
associazione - esempi vari



associazione : classi && oggetti – 1

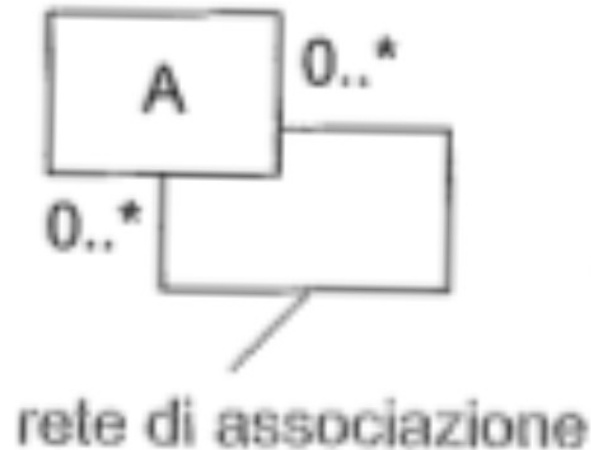


Esempio di diagramma degli oggetti

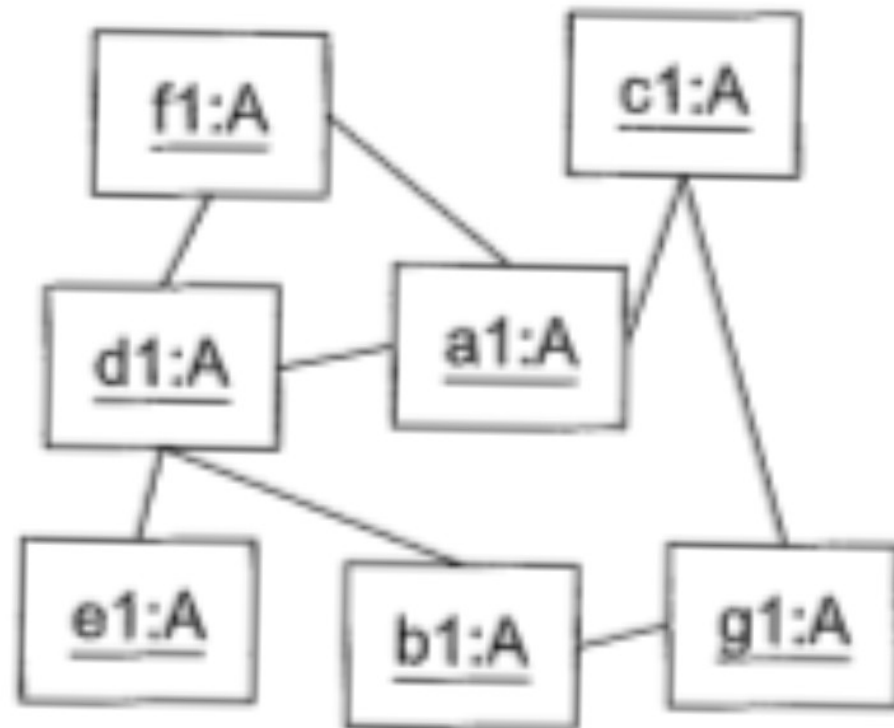


associazione : classi && oggetti – 2

Diagramma
delle classi



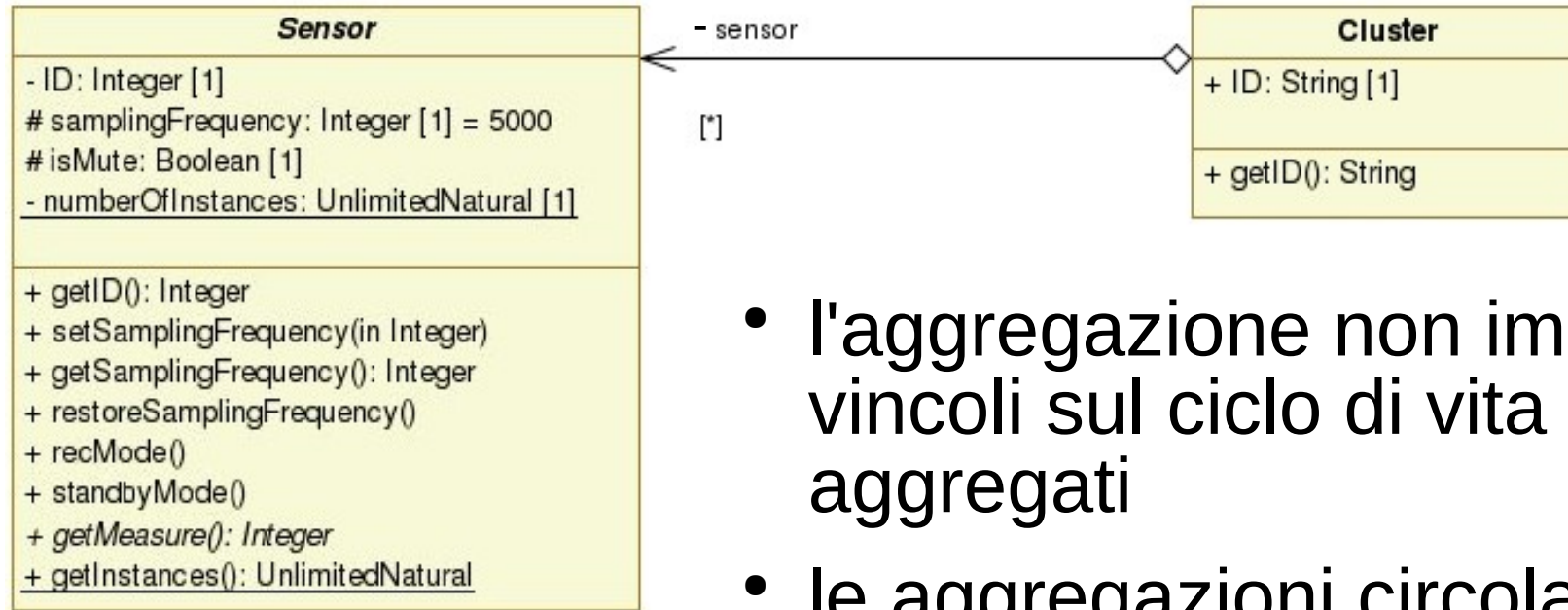
Esempio di diagramma degli oggetti



aggregazione

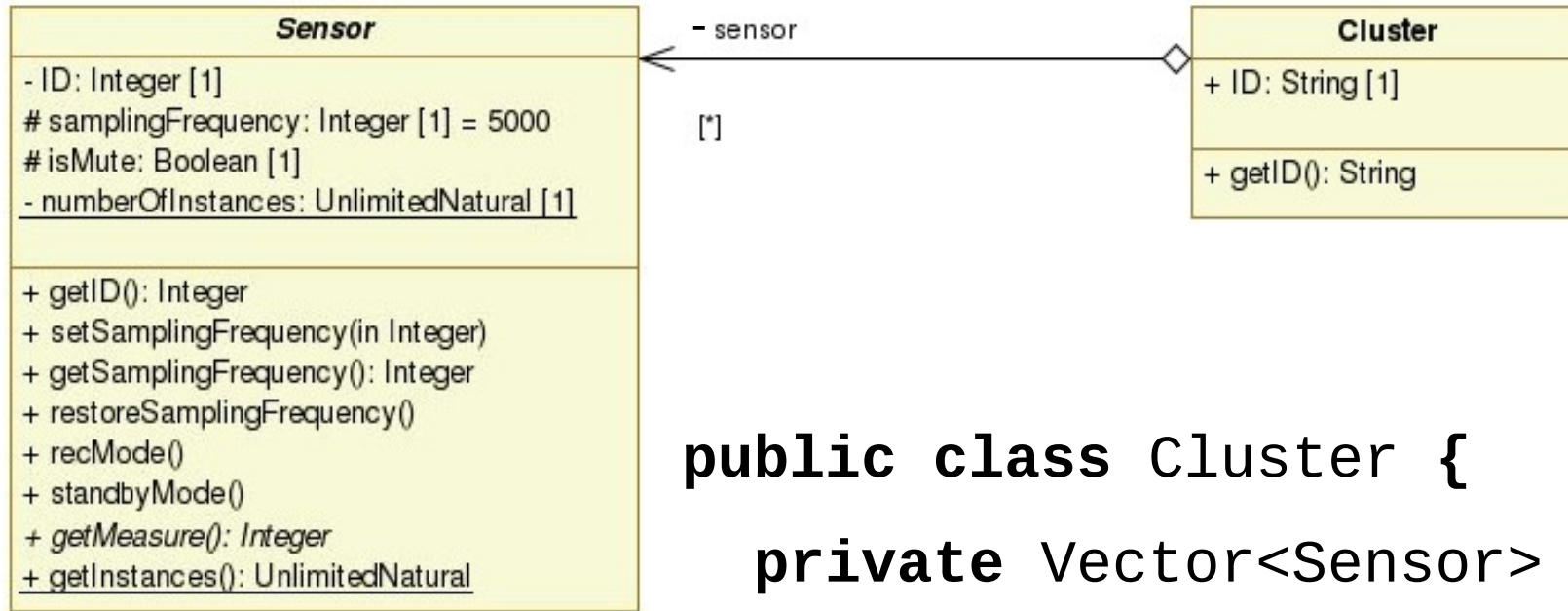
- è una relazione di tipo gerarchico :
 - una classe che rappresenta “entità autonoma”
 - una classe che aggrega l' “entità autonoma”
- esempio:
 - auto = motore + 4 ruote + scocca + ... *altre cose*
 - Paniere Titoli = $\sum$ Titolo_Azionario_i (*class view*)
 - FTSE MIB (Borsa Milano) = Enel + ENI + Telecom Italia + ... *altri titoli (instance view)*
- denominata anche: relazione “*whole-part*”
- rappresentata mediante una linea tra la classe aggregante e quella aggregata. all'estremità della classe aggregante, la linea ha un rombo bianco

aggregazione



- l'aggregazione non impone vincoli sul ciclo di vita degli aggregati
- le aggregazioni circolari sono da evitare perché (in base alla cardinalità) potrebbero portare a soluzioni **semanticamente errate**
 - A aggrega B
 - B aggrega C
 - C aggrega A

aggregazione – map in Java



```
public class Cluster {  
    private Vector<Sensor> vSensor;  
    public Cluster (Vector<Sensor> s){  
        this.vSensor= s;  
    }  
    public Cluster (){  
        this.vSensor= null;  
    }  
}
```

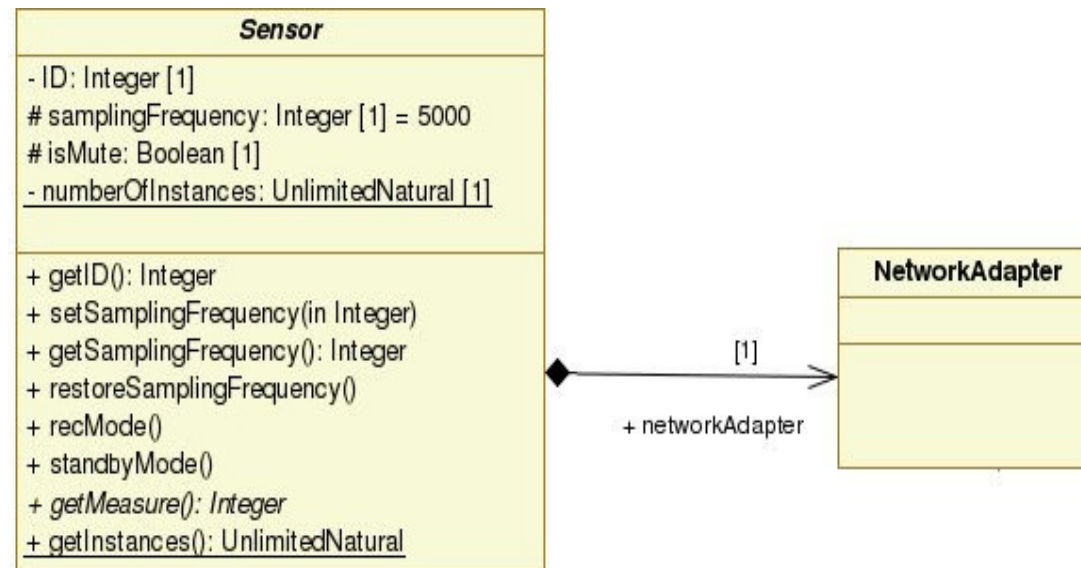
composizione

- è un'aggregazione forte
 - le istanze composte non devono necessariamente essere create con l'istanza della classe "componente"
 - ... ma una volta composte, le istanze delle componenti seguono il ciclo di vita dell'istanza che le compone
- esempio:
 - carbonara = spaghetti + uova + pancetta + ... altre cose
 - $\text{forum} = \sum \text{thread_di_discussione_i}$
 - $\text{thread_di_discussione} = \sum \text{commenti_i}$
- dipendentemente dall'ambiente di sviluppo:
 - la classe composita elimina le parti in un momento antecedente alla propria distruzione (dispose in C++)
 - l'implementazione è strutturata in modo che le istanze create con la composizione siano distrutte (con il garbage collector in Java)

composizione

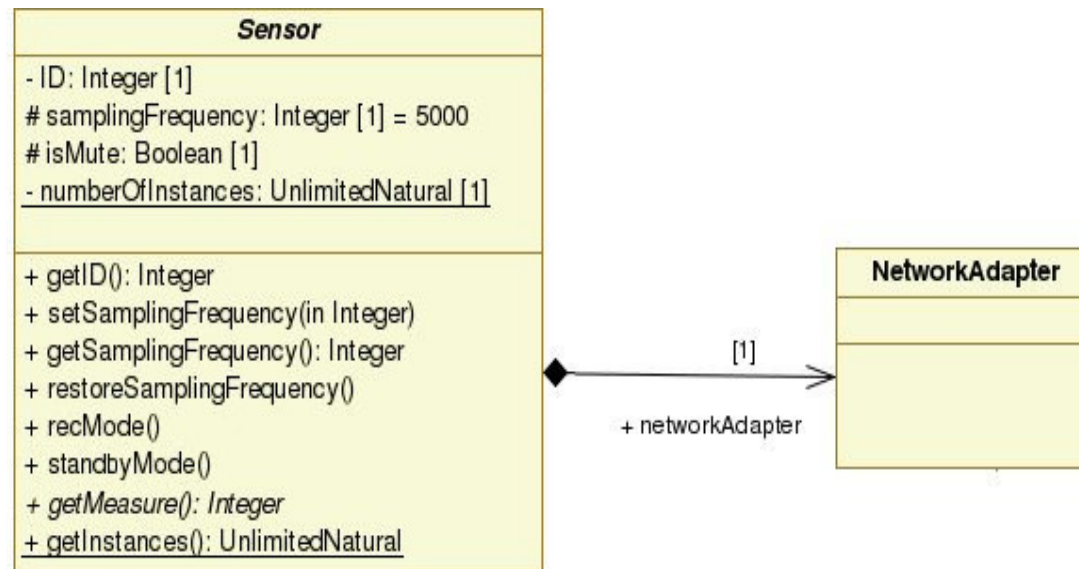
- conseguenze :
 - un oggetto può essere parte di un solo oggetto composto alla volta
 - invece nell'aggregazione una parte può essere condivisa tra più oggetti composti
- in UML è rappresentata mediante una linea tra la classe che compone (whole) e quella componente (part). all'estremità della classe che compone, la linea ha un rombo nero

composizione – map in Java – 1



composizione – map in Java – 2

```
public abstract class Sensor {  
    private NetworkAdapter na;  
  
    public Sensor () {  
        this.na = new NetworkAdapter();  
        ...  
    }  
    ...  
    /* Never pass  
    * the reference  
    * to this.na  
    * out of this  
    * class  
    */  
}
```



composizione – map in Java – 3

```
public abstract class Sensor {  
    private NetworkAdapter na;  
    public Sensor (NetworkAdapter net){  
        this.na = new NetworkAdapter();  
        /* Clone the state of net  
        * into this.na  
        */  
    }  
}
```

